

07

Model Quality and Evaluation

No single score can tell you whether a model is fit for purpose.

The four quality dimensions

Fitness

Can the model
replay observed
behavior?

Precision

Does it avoid
allowing unseen
behavior?

Generalization

Does it handle
plausible future
behavior?

Simplicity

Can people
understand and
use it?

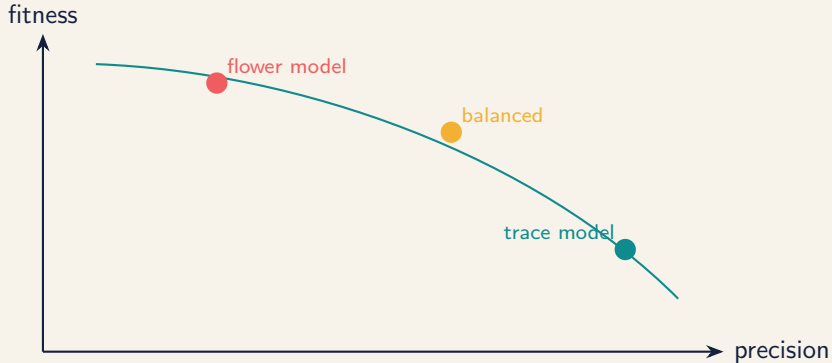
Key idea Perfect fitness is trivial if the model allows every possible trace. Quality emerges from the trade-off.

Build a quality vector, not one score

Dimension	Example measure	Interpretation risk
Fitness	Alignment / replay fitness	Rewards permissive models
Precision	Escaping-edge / alignment precision	Sensitive to log coverage
Generalization	Holdout replay, cross-validation	Depends on split design
Simplicity	Nodes, arcs, control-flow complexity	Small is not always understandable
Stability	Bootstrap edge / relation frequency	Aggregate scores can hide churn
Feasibility	Soundness, runtime, memory	Hard constraints, not preferences

Key idea Record metric direction, computation method, dataset split, and uncertainty for every component of the quality vector.

Fitness and precision pull in opposite directions



The “best” point depends on whether the model is for explanation, simulation, compliance, or prediction.

Why a weighted average can mislead

Suppose:

$$\text{score} = 0.4 \text{ fitness} + 0.4 \text{ precision} + 0.2 \text{ simplicity}$$

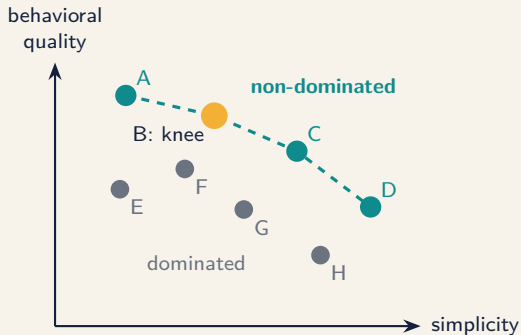
What it hides

- ▶ A critical minimum may be violated
- ▶ Different weights reverse the ranking
- ▶ Compensating metrics may be unacceptable
- ▶ Scale and normalization alter the result

Better first step

- ▶ Apply hard feasibility constraints
- ▶ Remove dominated candidates
- ▶ Inspect the Pareto front
- ▶ Add preferences only afterward

Pareto dominance and the Pareto front



Candidate X dominates Y when:

- ▶ X is no worse on every objective
- ▶ X is better on at least one objective

The **Pareto front** contains candidates not dominated by any other candidate.

Key idea A knee point offers a strong trade-off, but it is a candidate for discussion, not an automatic winner.

Practical Pareto-front selection

1. **Constrain first:** require soundness, minimum fitness, and an acceptable runtime or model size.
2. **Estimate uncertainty:** bootstrap cases or repeat temporal folds; do not trust tiny score differences.
3. **Inspect sensitivity:** vary thresholds, activity abstraction, and metric implementation.
4. **Review the front:** show all non-dominated models and explain what is gained and lost between adjacent candidates.
5. **Select transparently:** use a knee point, lexicographic rule, or stakeholder utility after the front is known.
6. **Validate once:** report the final choice on untouched holdout data.

Replay and alignments

Token-based replay

Replays traces on a Petri net and counts missing, remaining, consumed, and produced tokens.

Fast and diagnostic, but can mask some deviations.

Alignments

Find a minimum-cost correspondence between log behavior and model behavior.

More precise, but computationally more expensive.

Discuss Would you evaluate a compliance model and an exploratory model with the same tolerance for deviations? Why?

Evaluate candidates

```
fitness = pm4py.fitness_alignments(  
    log, net, initial, final  
)  
precision = pm4py.precision_alignments(  
    log, net, initial, final  
)  
  
print(fitness["log_fitness"])  
print(precision)
```

A defensible comparison records:

- ▶ Scores and runtime
- ▶ Model size and readability
- ▶ Training versus held-out performance
- ▶ Which behavior matters to the analytical question

A defensible evaluation protocol



- ▶ Candidate-generation rules and excluded candidates
- ▶ Full metric vectors with uncertainty, not only the selected model
- ▶ Pareto-front membership and the final decision rule
- ▶ Holdout performance, known instability, and intended model use

08

Conformance Checking

Find the smallest meaningful explanation of how reality and the model differ.

What counts as non-conformance?

The model may be normative

- ▶ Policy
- ▶ Regulation
- ▶ Standard operating procedure

A deviation may indicate risk.

The model may be descriptive

- ▶ Earlier discovery
- ▶ Expected operating pattern
- ▶ Simulation baseline

A deviation may indicate change.

Key idea Conformance identifies differences. Domain context determines whether those differences are harmful, helpful, or simply missing from the model.

Reading an alignment

Position	1	2	3	4	5
Log	Receive	Check	Clarify	Approve	Ship
Model	Receive	Check	»»	Approve	Ship
Move	sync	sync	log	sync	sync

Synchronous move

Event and model agree.

Move on log

Observed event not
matched.

Move on model

Expected behavior not
observed.

From deviations to findings

1. **Locate:** Which activities, paths, and cases deviate?
2. **Quantify:** How frequent, costly, or delayed are they?
3. **Segment:** Which products, teams, or periods differ?
4. **Validate:** Is the model current and the data complete?
5. **Explain:** What operational mechanism could produce this?

Strong finding

“Manual credit review is skipped in 7.3% of high-value orders, concentrated in one channel after the April system release.”

Alignment diagnostics in PM4Py

```
alignments = pm4py.conformance_diagnostics_alignments(  
    log, net, initial, final  
)  
  
for result in alignments[:3]:  
    print(result["fitness"])  
    print(result["alignment"])
```

Lab output: a ranked deviation table containing pattern, affected cases, frequency, performance impact, and a validated interpretation.